



Mobile Networks and IoT [2026-2027]

Assignment 02 – Using MQTT for communication

Introduction

This assignment aims to introduce the MQTT protocol for IoT networks. Students will interact with the protocol and create a simple program using MQTT for communication.

Objectives

Students successfully concluding this work should be able to:

- Understand the MQTT protocol
- Install a MQTT broker
- Use the MQTT protocol for communication

Requirements

- Raspberry Pi 2 Model B or Raspberry Pi Zero 2 WH as a MQTT broker (one per student's group)
- Mosquitto MQTT broker software
- Wireshark software (can be downloaded from <https://www.wireshark.org>)
- Python installed in student's PC

Support Material

- Example code
- Class slides
- See References section and Annex for additional information

Notes

- In some systems you might need to use *python3* instead of *python*.
- Sometimes, students have multiple versions of Python installed on their computer, which can lead to problems when using MQTT libraries. A common issue is installing libraries in one version of Python while running code with another.

- To connect to the given MQTT broker located at DEI-FCTUC, use Wi-Fi SSID “DEI” and your login account at dei.uc.pt.
- Homework exercises (optional) require an MQTT server. You can use any internet or local MQTT server. An additional MQTT server will also be available at DEI, which you can access from home via DEI's VPN. Ask your teacher for more information.

Background

This section covers some concepts that will be used during MQTT assignments.

The MQTT protocol

The Message Queue Telemetry Transport (MQTT) is a lightweight and simple messaging protocol targeted specifically for IoT applications [1]. MQTT uses a publish-subscribe methodology, where clients send and receive messages to each other through a centralized *broker*¹, also sometimes called a *server*. Clients both publish and subscribe to information channels called *topics*, and any data that passes on via the broker is tagged with a topic label. Once clients are pointed to the broker's IP address, there's no more system configuration involved. Clients simply send messages to a topic (that may or may not exist elsewhere) by publishing topic-tagged data to it. Clients listen to topics by subscribing to them [2].

While a Raspberry Pi is powerful enough to leverage more complex messaging protocols, in distributed IoT solutions, it is likely to use MQTT as messaging protocol. The simplicity and open nature of MQTT makes it easy to learn and use [1].

Client-Server communication

When two computers want to communicate to each other, and unless they are using a direct connection, they often use a server. The typical configuration is depicted in Figure 1.

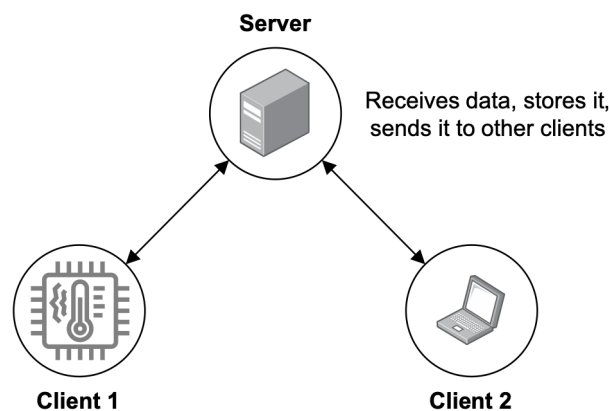


Figure 1. Typical client/server configuration (adapted from [3]).

In Figure 1, Client 1 sends data, and the server saves it. Then, when Client 2 sends a request for that data, the server sends it. An example based on IoT could be as follows:

- Client 1 reads a temperature value and sends it to the server;
- The Server receives it and stores it;
- Client 2 requests the data from the server;
- The Server replies with the requested data;
- Client 2 receives the temperature data.

¹ A *broker* is an entity that arranges or negotiates an agreement

Publisher/Subscriber communication

The communication using MQTT is slightly different than the typical case. First, in MQTT there is no need to store the data at the server, as this step is optional. Instead, the server immediately sends the data to any client who needs it. The server identifies the clients that need the data using the concept of **topics**. Using an IoT example, a topic can be a sensor value, or a parameter sent by one or more devices. A useful analogy is TV or radio. A TV broadcaster broadcasts a TV program using a specific channel and a viewer tunes into this channel to view the broadcast. There is no direct connection between the broadcaster and the viewer [4]. MQTT uses a **publish-subscribe architecture**. Figure 2 shows a schematic view of communication using a MQTT.

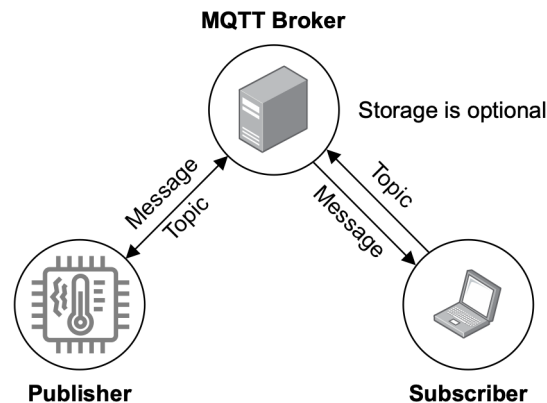


Figure 2. Communication based on MQTT.

MQTT uses the publish/subscribe architecture to decouple the publisher (message sender) from the subscriber (message receiver). A third component, called a message broker, handles the communication between publishers and subscribers. The broker's job is to filter all incoming messages from publishers and distribute them correctly to subscribers, using the concept of topic. Also, the broker decouples publishers and subscribers in space, time, and synchronization. **Space decoupling** means that publisher and subscriber are not aware of each other's network location. **Time decoupling** happens as publishers and subscribers do not need to run or have network connectivity at the same time. **Synchronization decoupling** because publishers and subscribers can send or receive messages without interrupting each other.

Subscribe a topic using paho-mqtt:

```
client.subscribe(MQTT_TOPIC)
```

Publish a message using paho-mqtt:

```
client.publish(MQTT_TOPIC, "Hello World!")
```

Retained MQTT messages

An MQTT broker can store a retained message that can be sent immediately upon a new MQTT subscription. This way new clients receive a message immediately after subscribing. The broker stores only one retained message per topic (the last retained message published). By enabling the retention of the most recent message on a topic, subscribers can stay informed about the current state, even during periods of inactivity.

Publish a retained message using paho-mqtt:

```
client.publish(TEMP_TOPIC, "22.5", qos=1, retain=True)
```

To delete the retained message of a topic, simply publish a retained message with a zero-byte payload to the topic where the retained message is stored.

MQTT Last Will

When a publisher client connects to a MQTT broker, it can specify a message to be sent in case it ungracefully disconnects. This allows for a more graceful system wide notification that a publisher client has been disconnected. The Last Will allows clients to notify others about their unexpected disconnections. The broker broadcasts the Last Will message to all subscribed clients of the corresponding topic. The broker discards the stored Last Will message if the client disconnects gracefully using the DISCONNECT message.

Send a Last Will message using paho-mqtt:

```
client.will_set(STATUS_TOPIC, "Sensor offline", qos=1, retain=True)
```

MQTT QoS level

MQTT implements 3 Quality of Service (QoS) levels:

- QoS 0: At most once (best-effort and unacknowledged data service)
- QoS 1: At least once (ensures message delivery)
- QoS 2: Exactly once (neither loss nor duplication)

These QoS levels allow for more reliable IoT applications, being able to adapt to unreliable network conditions, guaranteeing that data is delivered from the sender to the receiver through the MQTT broker.

QoS is an attribute of an individual message being published. A subscribing client can also set the maximum QoS a server uses to send messages that match the client subscriptions. The QoS of a message forwarded to a subscriber thus might be different to the QoS given to the message by the original publisher. The lower of the two values is used to forward a message.

The QoS on publishing and subscribing defaults to 0.

Broker and Python library

In this assignment we will use the Eclipse Mosquitto [5], an open-source MQTT broker, and paho-mqtt2.1.0 [6], the Eclipse Paho MQTT Python client library. The use of mqtt1.6.1 [7] should be avoided as it is already deprecated.

How to install an MQTT broker and libraries

(If an MQTT broker is provided skip to step 0)

1. Install both the MQTT broker and its client tools on the Raspberry Pi

First, update and upgrade your system (update the list of available packages and then install new versions of the used packages):

Note: skip next command if using the class Raspberry Pi, which is already up to date.

```
sudo apt update && sudo apt upgrade
```

On the Raspberry Pi, first install the Mosquitto MQTT broker with the command below:

```
sudo apt install -y mosquitto mosquitto-clients
```

This command will install the **Mosquitto MQTT broker** and client libraries on the Raspberry Pi. The broker is now installed on the Raspberry Pi. To run the MQTT broker every time the Raspberry Pi boots, use the following command:

```
sudo systemctl enable mosquitto
```

(If you want to run only this time, use `sudo systemctl start mosquitto`)

To confirm the Mosquitto version, use the following command:

```
mosquitto -h
```

To confirm that Mosquitto is running, use the following command:

```
sudo systemctl status mosquitto
```

By default, Mosquitto only accepts local connections (clients running on the same machine). To enable remote access the configuration file must be changed:

```
sudo nano /etc/mosquitto/mosquitto.conf
```

Add the following lines at the end of the file to enable remote access and clients to connect without authentication:

```
listener 1883 0.0.0.0
allow_anonymous true
```

(Your `mosquitto.conf` file should now look like the one in Annex B.)

This will allow incoming TCP connections on port 1883 from any of your device's interfaces and allows anonymous clients. For enhanced security (not used in this assignment) authentication should be enabled. Restart Mosquitto MQTT server to apply changes:

```
sudo systemctl restart mosquitto
```

2. Test the Raspberry Pi Mosquitto installation

You will use your Raspberry Pi as publisher and subscriber for this test. Open a terminal window and run the subscriber command first:

```
mosquitto_sub -h <IP_address> -t "topic"
```

The `-h` option identifies the host (use your Raspberry Pi's IP address or the string **localhost**), the `-t` option identifies the topic. On a second terminal window, run the publisher command:

```
mosquitto_pub -h <IP_address> -t "topic" -m "message"
```

The `-m` option identifies the message that you want to publish. Use the same topic as your subscriber. If everything is working correctly, you should see the message string in your subscriber terminal window.

Note: if you copy&paste directly from this pdf, you may get the wrong quotation marks, change them.

Type Ctrl+C in the subscriber terminal window to exit subscriber.

3. Install Python libraries on the Raspberry Pi

To be able to use Python as the development language for MQTT on the Raspberry Pi, MQTT Python libraries must be installed. The most popular MQTT library for Python is `paho-mqtt` library. Before continuing make sure that Python 3.7+ is installed:

```
python --version OR python3 --version
```

If it is not installed, install it. It is necessary to work with the latest `paho-mqtt` libraries.

To install `paho-mqtt` library on the Raspberry Pi use the command:

```
sudo pip3 install paho-mqtt
```

Note: if command fails, try: `sudo apt install python3-paho-mqtt`

4. Install the MQTT Python libraries on your PC

To prepare the PC environment to use Python as the development language for MQTT using `paho-mqtt`, `paho-mqtt` libraries need to be installed. After the installation, you will be able to use Python to send and receive data from the MQTT broker running on the Raspberry Pi. It can be installed by running the command below:

```
pip3 install paho-mqtt
```

Note: In Microsoft Windows make sure to run the command prompt by right clicking and choosing *"Run as Administrator"*, otherwise it may cause issues during installation.

Exercises

1. Publishing and subscribing data using MQTT and 1 PC

Goal: understand the basics of publishing and subscribing data using MQTT.

This exercise will use the Mosquitto MQTT server on the Raspberry Pi and the student's PC.

- Download and analyze the files that are included with this assignment. There are versions of subscribers and publishers for paho-mqtt 1.6.1 (deprecated) and paho-mqtt 2.1.0+.
- Adapt the variable MQTT_BROKER and MQTT_TOPIC to your specific server and topic. You can also update the message you want to publish (the example is sending the string *"Hello World!"*).
- Use the Raspberry Pi as a broker and a PC as publisher/subscriber (use different terminals for each one) to share data via MQTT (Figure 3).

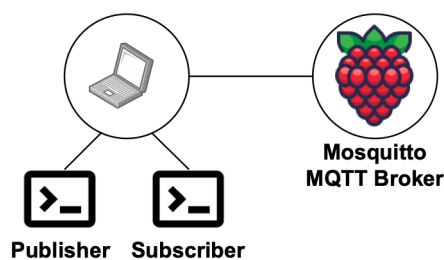


Figure 3. Class setup.

Run the publisher code on your PC:

```
python publisher.py 2
```

Run the subscriber code on your PC:

```
python subscriber.py 2
```

You should receive the messages sent by the publisher (*"Hello World!"* or any other message that you have configured).

2. Using Wireshark to analyse the MQTT packets

Goal: Use Wireshark to analyse and debug MQTT packets that are exchanged in the network.

In this exercise you will use Wireshark³ to identify MQTT packets. Wireshark is a network sniffer and packet analyzer used for network troubleshooting. By using a network sniffer such as Wireshark, the messages between client and server can be seen. The information available includes the packet header and the data.

Use Wireshark to analyse the previous built scenario (Figure 3), focusing on the exchanged MQTT packets. Use filters to facilitate the analysis. For example, "ip.addr" matches against both the IP source and destination addresses in the IP header. Use `ip.addr=={IP}` to show only

² You might need to use python3 instead of python

³ <http://www.wireshark.org>

packets from your PC or to the MQTT broker (e.g. `ip.addr==10.6.2.20`). To find your IP use the network commands from previous assignments.

Find at least one message from the publisher to the broker and one from the broker to the subscriber. Identify the topic used and messages sent. Your capture should be like the one displayed in Figure 4.

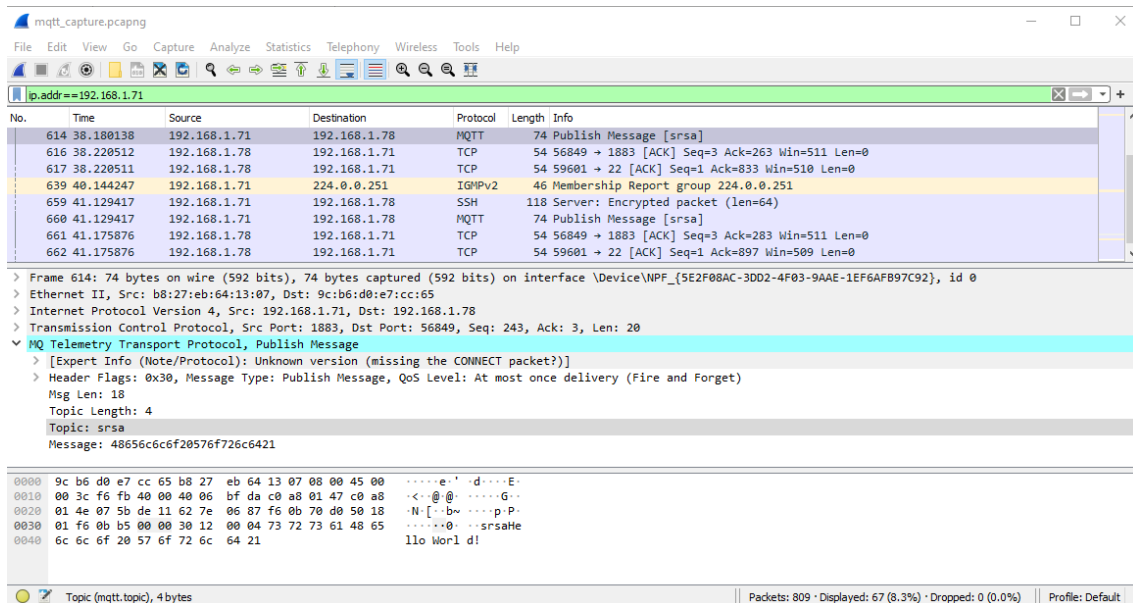


Figure 4. Sample MQTT capture.

With Wireshark you can identify all the headers and contents of your packet, split by the TCP/IP architecture layer. Identify fields such as the source and destination IP address, the ports used, the message sent.

3. Using hierarchical topics, retained messages and last will

Goal: use hierarchical topics, MQTT retained messages and MQTT last will.

In this exercise the given files (already used in Exercise 1) will be adapted to create more with the following guidelines:

- Create 3 subscribers and 2 publishers
- One publisher publishes to topic “home/temperature” and the other to topic “home/humidity”
- The first subscriber subscribes to the “home/temperature” topic, the second to the “home/humidity” topic, and the third to both.
- Each publisher creates a new message with its data every 1 to 5 seconds and publishes it. The measurement value can be random.
- Use wildcards to select the topics to subscribe whenever possible.
- Start with a QoS level = 0.
- Use retained messages so that new subscribers always receive the last known temperature value (humidity does not have retained messages).

- Set a Last Will message so that if a publisher disconnects unexpectedly, a message with “-1” is published to the topic.

Exercise steps

- Create the Python scripts that solve the exercise
- Test all scripts by running each in a different terminal window
- If you notice that some of the messages are lost, change the QoS level when publishing and subscribing.
- Simulate an ungracefully publisher disconnect (e.g., pressing Ctrl+C). Check if Last Will is received by subscribers. (**Note:** A try-except should not exist in the sensors script that makes a clean disconnect on receiving Ctrl+C)
- Disconnect a subscriber and reconnect again. Is the retained temperature received?
- Verify in Wireshark that messages in JSON are being sent.

4. Using MQTT with multiple topics, publishers, and subscribers (homework)

Goal: Use MQTT with multiple topics, publishers, and subscribers.

This exercise will use the Mosquitto MQTT server on the Raspberry Pi and PCs from different students that will subscribe and publish on different topics.

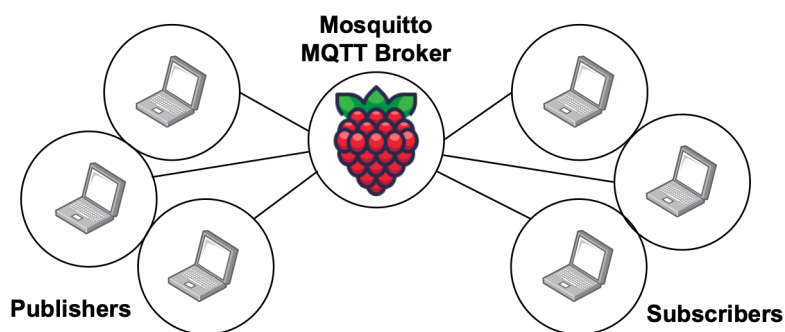


Figure 5. Class setup for Exercise 4.

The goal of the exercise is to create a simple chat using MQTT. Each PC, belonging to a different student, will publish to and subscribe from different topics, using the MQTT broker located on the Raspberry Pi. Adapt the files to be able to send/receive messages to/from multiple topics.

Program requirements:

- The user must have 3 commands: subscribe, unsubscribe, send message, and quit
- Command “subscribe”: subscribes a specific topic
- Command “unsubscribe”: unsubscribes a specific topic
- Command “send”: asks for a message and topic and sends it

Exchange messages with other students using the program created.

5. Using MQTT with multiple servers (homework)

Goal: Use MQTT with multiple servers, topics and subscribers.

Adapt the solution of the previous exercise to enable the use of different MQTT brokers. Each student PC will act as a publisher and subscriber, sending and receiving messages to a specific topic located at a specific broker. Both broker address and topic are given by command-line parameters.

References

- [1] G. Smart, Practical Python Programming for IoT: Build advanced IoT projects using a Raspberry Pi 4, MQTT, RESTful APIs, WebSockets, and Python 3, Packt Publishing, 2020.
- [2] "MQTT Tutorial: Setup MQTT Server on Raspberry Pi," Arrow Electronics, [Online]. Available: <https://www.arrow.com/en/research-and-events/articles/mqtt-tutorial>.
- [3] "MQTT Communication between Raspberry Pi and PC using Paho," Kitflix Technologies, [Online]. Available: <https://kitflix.substack.com/p/communication-between-raspberry-pi-and-pc-using-mqtt>.
- [4] "Practical Guide to MQTT and Mosquitto," Steve's Internet Guide, [Online]. Available: <http://www.steves-internet-guide.com/mqtt-works/>.
- [5] E. Foundation, "Eclipse Mosquitto," [Online]. Available: <https://mosquitto.org>. [Accessed October 2025].
- [6] "paho-mqtt 2.1.0," [Online]. Available: <https://pypi.org/project/paho-mqtt/>. [Accessed October 2025].
- [7] "paho-mqtt 1.6.1," [Online]. Available: <https://pypi.org/project/paho-mqtt/1.6.1/>. [Accessed October 2025].

ANNEX

A – Eclipse Paho MQTT Python client library

This library implements versions 5.0, 3.1.1, and 3.1 of the MQTT protocol.

Overview and examples at:

<https://pypi.org/project/paho-mqtt/>

Full documentation available at:

<https://eclipse.dev/paho/files/paho.mqtt.python/html/index.html>

Description of some of the Python MQTT library functions used

Publish

Single

Publish a single message to a broker, then disconnect cleanly.

Example:

```
import paho.mqtt.publish as publish
publish.single("paho/test/topic", "payload", hostname=
"mqtt.eclipseprojects.io")
```

Multiple

Publish multiple messages to a broker, then disconnect cleanly.

Example:

```
from paho.mqtt.enums import MQTTProtocolVersion
import paho.mqtt.publish as publish
msgs = [{'topic': "paho/test/topic", 'payload': "multiple 1"},
("paho/test/topic", "multiple 2", 0, False)]
publish.multiple(msgs, hostname="mqtt.eclipseprojects.io",
protocol=MQTTProtocolVersion.MQTTv5)
```

Subscribe

The two functions provided are `simple()` and `callback()`. Both functions include support for MQTT v5.0, but do not currently let you set any properties on connection or when subscribing.

Simple - Subscribe to a set of topics and return the messages received. This is a blocking function.

Example:

```
import paho.mqtt.subscribe as subscribe
```

```
msg = subscribe.simple("paho/test/topic", hostname=
"mqtt.eclipseprojects.io")
print("%s %s" % (msg.topic, msg.payload))
```

Using Callback - Subscribe to a set of topics and process the messages received using a user provided callback.

Example:

```
import paho.mqtt.subscribe as subscribe

def on_message_print(client, userdata, message):
    print("%s %s" % (message.topic, message.payload))
    userdata["message_count"] += 1
    if userdata["message_count"] >= 5:
        # it's possible to stop the program by disconnecting
        client.disconnect()

subscribe.callback(on_message_print, "paho/test/topic",
hostname="mqtt.eclipseprojects.io", userdata={"message_count": 0})
```

Unsubscribe

```
unsubscribe(topic)
```

Unsubscribes the client from one or more topics, where `topic` is a single string, or list of strings that are the subscription topics to unsubscribe from. Returns a tuple (result, mid), where result is `MQTT_ERR_SUCCESS` to indicate success, or (`MQTT_ERR_NO_CONN`, `None`) if the client is not currently connected. mid is the message ID for the unsubscribe request.

B) Default /etc/mosquitto/mosquitto.conf

```
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example

pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto/

log_dest file /var/log/mosquitto/mosquitto.log
include_dir /etc/mosquitto/conf.d

listener 1883 0.0.0.0
allow_anonymous true
```